Firewall Log Classification Using Machine Learning

CSAI 801 – Artificial Intelligence & Machine Learning
Yasmeen Algendy, Yomna Algendy, Zahraa Mohamed
Supervisor: Dr. Marwa Elsayed

Abstract

The increasing volume of network traffic makes manual firewall monitoring impractical. This project automates the classification of firewall log actions into four categories: *allow*, *drop*, *deny*, and *reset-both* using machine learning. The dataset (65,532 records, 12 features) was preprocessed through duplicate removal, outlier filtering, and feature scaling. SMOTE oversampling addressed class imbalance in the training set. Three models were evaluated: Random Forest, logistic regression, and K-Nearest Neighbors (KNN). The tuned Random Forest achieved 99.89% accuracy, while Logistic Regression reached 99.75%, demonstrating that machine learning can reliably support automated network security monitoring.

1 *Introduction*

Enterprise firewalls produce massive log volumes that are impractical for manual review, which is slow and error prone [1]. Machine learning enables automated, near real-time classification of logs, highlighting anomalies for analysts [5]. This project compares Random Forest, Logistic Regression, and KNN on a real firewall log dataset, leveraging their respective strengths [3, 4]. The goal is to build a classifier that accurately distinguishes four firewall actions with ≥95% accuracy for automated monitoring.

1.1 Enhancing Project Performance with Research-Based Methods:

In this project, we applied the Random Forest model to classify firewall logs and further enhanced its performance through hyperparameter tuning. The baseline model initially achieved 98.3% accuracy, which improved to 99.89% after tuning. This optimization not only increased overall accuracy but also strengthened minority-class detection, demonstrating the benefits of adopting best practices from prior research. [5].

2 *Method*

2.1 *Dataset*

The dataset consists of 65,532 firewall log records from the UCI Machine Learning Repository [2] with 12 numerical features: *Source Port*, *Destination Port*, *NAT Source Port*, *NAT Destination Port*, *Bytes*, *Bytes Sent*, *Bytes Received*, *Packets*, *Elapsed Time*, *pkts sent*, and *pkts received*. The target variable *Action* encodes four classes: allow (32,476), deny (12,899), drop (11,081), and reset-both (565).

2.2 *Data Preprocessing*

Removed 8,532 duplicates, about 13 percent, and discarded extreme values using IQR, leaving 29,599 records. Categorical targets were encoded as integers with allow 0, drop 1, deny 2, and reset-both 3. A stratified 70/30 train/test split produced 20,720 training and 8,879 test records while preserving class distribution. SMOTE was applied on the training set to balance classes, and features were normalized using StandardScaler for KNN and Logistic Regression.

2.3 *Models*

Logistic Regression: Serves as the primary linear baseline. Multi-class probabilities are estimated via softmax, and coefficients indicate feature importance.
k-Nearest Neighbors (KNN): Classifies instances by majority vote among neighbors. Being non-parametric and sensitive to irrelevant features, proper feature scaling is essential.
Random Forest: An ensemble of decision trees trained on bootstrap samples with random feature subsets. A baseline model was evaluated, then tuned via 5-fold GridSearchCV over the hyperparameter grid.

3 *Experiments*

All experiments followed a consistent protocol: models were trained on the SMOTE-augmented training set, evaluated on the held-out test set, and additionally assessed with 10-fold stratified cross-validation to measure variance in generalisation performance.

Evaluation Metrics:

Accuracy, per-class Precision, Recall, F1-score, Log Loss, and Confusion Matrix were recorded for every model.

Baseline Comparison:

The three baseline models were run with default or lightly configured hyperparameters. Fig. 1 compares their train and test accuracy, while Fig. 2 shows their confusion matrices.

Figure 1: Model comparison before tuning (train vs. test accuracy).

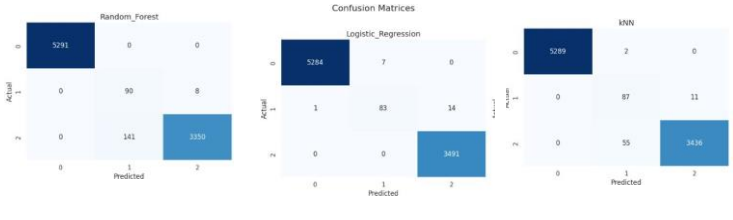

Figure 2: Confusion matrices before tuning.

Hyperparameter Tuning:

Given Random Forest's competitive baseline performance, `GridSearchCV` was applied to find the optimal configuration (Table 1).
Fig. 3 shows the accuracy of improvement after tuning, and Fig. 4 confusion matrix.

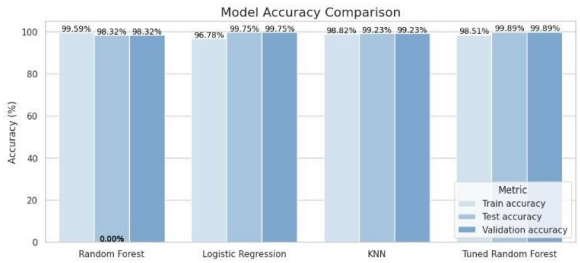

Figure 3: Model accuracy comparison after hyperparameter tuning.

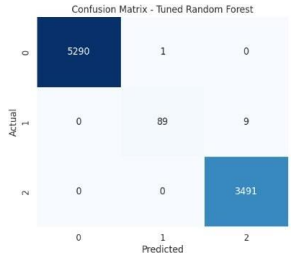

Figure 4: Confusion matrix for the tuned Random Forest.

SMOTE Ablation:

Early experiments without SMOTE showed near-zero recall for the *reset-both* class and degraded recall for *drop* and *deny*, confirming that oversampling was essential for balanced detection.

Feature correlation analysis revealed that *Bytes*/*Packets* (r = 0.97), *Bytes Sent*/*pkts sent* (r = 0.97), and *Bytes Received*/*pkts received* (r = 0.95) are nearly redundant. All features were retained; future work may explore dimensionality reduction.

Feature Correlation Analysis:

Some features were highly correlated, such as Bytes with Packets and Bytes Sent with pkts_sent, but all were retained for this study. Future work could explore PCA-based dimensionality reduction to reduce computational cost.

Error Analysis:

Most errors came from the minority deny class, with the baseline Random Forest misclassifying many deny samples as drop. The tuned Random Forest drastically reduced these errors, showing that limiting depth improves generalization. Logistic Regression made a few mistakes mostly involving allow and deny classes, highlighting the challenge of predicting minority classes accurately.

4 *Results*

Model	Test Accuracy	Macro F1
kNN	99.23%	0.990
Logistic Regression	99.75%	0.997
Random Forest (base)	98.32%	0.981
Random Forest (tuned)	**99.89%**	**0.999**

Table 2: Model accuracy on the held-out test set.

The tuned Random Forest achieved the best performance with **99.89% accuracy**, followed by Logistic Regression at **99.75%**. It also produced the **fewest misclassifications**, especially for the drop and deny classes, and achieved **AUC = 0.98** for the minority class.

5 Limitations and Future Work

The current system is trained offline; deployment would require a retraining pipeline to adapt to evolving threat patterns. Highly correlated features suggest that dimensionality reduction via PCA may reduce computational overhead without sacrificing accuracy. Future work should also explore deep learning architectures (e.g., LSTMs for temporal log sequences), ensemble stacking, and evaluation on additional firewall datasets to assess generalizability.

6 Application Demo

An interactive web interface was developed using Streamlit to allow users to test the trained models on firewall log data. The application enables real-time prediction and visualization of results.

Huggingface link: Log Classifier

7 Conclusion

This project demonstrates that machine learning can effectively automate firewall log classification, surpassing the 95% accuracy target. The tuned Random Forest achieved 99.89% accuracy and Macro F1 of 0.999, while Logistic Regression reached 99.75%. The baseline Random Forest struggled with the minority deny class (F1 = 0.55) compared to 0.95 for the tuned model, showing the importance of proper evaluation. Success relied on robust preprocessing including SMOTE for minority-class detection, stratified splits, and consistent label encoding, while hyperparameter tuning and feature scaling allowed linear models to compete with tree-based ensembles

References

[1] *H. As-Suhbani and S. D. Khamitkar, "Classification of firewall logs using supervised machine learning algorithms,"*
Int. J. Comput. Sci. Eng., vol. 7, no. 6, pp. 456–462, 2019.

[2] *UCI Machine Learning Repository, Internet Firewall Data, University of California, Irvine. Online. Available: https://archive.ics.uci.edu/dataset/542/internet+firewall+data*

[3] *M. Ali, M. F. Mushtaq, U. Akram, S. Ramzan, S. Tahir, and M. Ahsan, "An ensemble approach for firewall log classification using stacked machine learning models," J. Comput. Biomed. Inform., vol. 5, no. 1, pp. 112–128, 2025.*

[4] *B. Al-Tarawneh and H. Bani-Salameh, "Classification of firewall logs actions using machine learning techniques and deep neural network," EasyChair Preprint, no. 7845, 2022.*

[5] *M. Aljabri, A. A. Alahmadi, R. M. A. Mohammad, M. Aboulnour, S. M. Aldossary, M. S. Al-Ghamdi, and*
F. Aljurbua, "Classification of firewall log data using multiclass machine learning models," Electronics, vol. 11, no. 9, p. 1414, 2022.
https://www.mdpi.com/2079-9292/11/12/1851

[6] *N. V. Chawla, K. W. Bowyer, L. O. Hall, and W. P. Kegelmeyer, "SMOTE: Synthetic minority over-sampling technique," J. Artif. Intell. Res., vol. 16, pp. 321–357, 2002.*

[7] *L. Breiman, "Random forests," Mach. Learn., vol. 45, no. 1, pp. 5–32, 2001.*